

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl
Informatik, Universität Trier

Organisatorisches

- **Vorlesung:** Di, 14-16 Uhr c.t. und Do, 12-14 Uhr c.t. in H 507
- **Übung:** Di 16-18 → Donnerstags, 14-16 Uhr s.t. in H507 ????
- Melden Sie sich in PORTA zur Veranstaltung an
 - Vorlesung: **14803172**
 - ~~Übung: 14803171~~
- **Material zur Vorlesung finden Sie in Stud.IP**
 - Vorlesungsfolien, Übungsblätter, Informationen zu den Projekten
- **Abgabe der Übungen und Projekte erfolgt in Stud.IP**

Organisatorisches

- **Literatur:**

- Reinhard Wilhelm, Dieter Maurer: **Übersetzerbau**, *Springer-Verlag*, 1992
- Karen Lemone: **Design of Compilers - Techniques of Programming Language Translation**, *CRC Press*, 1992
- Stephan Diehl: **A Formal Introduction to the Compilation of Java**, *SP&E, Wiley*, 1998
- Keith D. Cooper, Linda Torczon: **Engineering a Compiler**, *Morgan Kaufmann*, 2004
- Torben Ægidius Mogensen: **Introduction to Compiler Design**, *Springer*, 2011
- Alfred Aho, Monica Lam, Ravi Sethi, Jeffrey Ullman: **Compiler - Prinzipien, Techniken und Werkzeuge**, *PEARSON Studium*, 2008

Übersetzung und Analyse von Programmen

- **Inhalt:**

- Semantik von Programmiersprachen
- Überblick über verschiedene Phasen eines Compilers
- Lexikalische und syntaktische Analyse
- Statische Programmanalyse
 - insbesondere Datenflussanalyse
- Übersetzung von Java
- Abstrakte Maschinen insbesondere JVM
- Hybride Analysen
 - Erkennung von Code-Klonen
 - Erkennung von Entwurfsmustern
- Programmtransformationen
 - Programmoptimierung
 - Partielle Auswertungen

Vorlesungsbegleitendes Projekt:

- Implementierung der TRAM
- Übersetzung von TRIPLA

Prüfungsleistung

- **Portfolio**

- 5 aus 7 Übungen (10 Punkte pro Übungsblatt)

50

- 5 Teilprojekte

- TRAM (10)

80

- Lexer+Parser (20)

- Code-Erzeugung (25)

- Kontrollflussanalyse (5)

- Datenflussanalyse (20)

130

- **Abgabe des Portfolio bis zum 17.3.2026**

Geplanter Ablauf von Vorlesung, Übung und Projekt

6

Datum	Thema	Übung	Projekt
14.10.2025	Einleitung		
16.10.2025	Abstrakte Maschinen	1. Übungsblatt ÜAM 10P	
21.10.2025	TRAM		Projekt I (TRAM) 10P
23.10.2025	Hinweise zur Implementierung von Projekt I	Abgabe ÜAM	
28.10.2025	Lexikalische Analyse (Teil 1) Beispiele, RE->NFA	2. Übungsblatt ÜLEX 10P	
30.10.2025	Lexikalische Analyse (Teil 2) NFA->DFA		Abgabe Projekt I
04.11.2025	Syntaktische Analyse (Teil 1)		Projekt II (Lexer+Parser) 20P
06.11.2025	Syntaktische Analyse (Teil 2)	Abgabe ÜLEX	
11.11.2025	Hinweise zur Implementierung von Projekt II, PLY	3. Übungsblatt ÜSYN 10P	
13.11.2025	Semantik von Programmiersprachen		
18.11.2025	Semantik von TRIPLA		
20.11.2025	Übersetzung von TRIPLA		
25.11.2025	Übersetzung von Java (Teil 1)	Abgabe ÜSYN	
27.11.2025	Hinweise zur Implementierung von Projekt III	4. Übungsblatt ÜCETRIPLA 10P	Abgabe Projekt II
02.12.2025	Übersetzung von Java (Teil 2)	Blatt ÜCETRIPLA	Projekt III (Codererzeugung) 25P
04.12.2025	Übersetzung von Java (Teil 3)		
09.12.2025	Just-in-Time Übersetzung von Java	Abgabe ÜCETRIPLA + 5. Übungsblatt ÜCEJAVA 10P	
11.12.2025	Abstrakte Interpretation (von TRIPLA)		
16.12.2025	Übersetzung mittels LLM (Matthias Wölwer)	Abgabe ÜCEJAVA	
18.12.2025			
	22.12.2025 - 07.01.2026 FREI		
08.01.2026	Datenflussanalyse	6. Übungsblatt ÜDFA 10P	
13.01.2026	Datenflussanalyse von TRIPLA		Projekt IV (Kontrollflussanalyse) 5P
15.01.2026	Hinweise zur Implementierung von Projekt IV		
20.01.2026	Code Clone Detection	Abgabe Blatt ÜDFA	
22.01.2026	Reverse Engineering, Erkennung von Entwurfsmustern		Abgabe Projekt IV
27.01.2026	Program Analysis and Transformation from a Practitioner's Point of View (Jonas Eckstein, itestra)		Projekt V (Datenflussanalyse) 20P
29.01.2026	Programmtransformationen/-optimierungen		
03.02.2026	Partielle Auswertung von TRIPLA	7. Übungsblatt ÜTRAFO 10P	
05.02.2026	Details zum Portfolio & Hinweise zur Implementierung von Projekt V		
10.02.2026			
12.02.2026		Abgabe ÜTRAFO	

Compiler und Interpreter

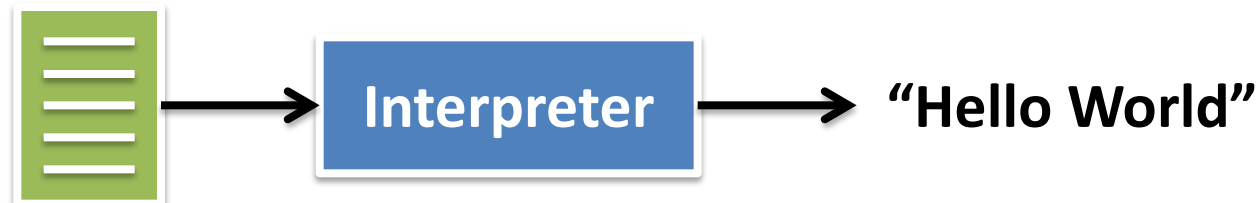
- **Was ist ein Compiler?** [laut Cooper&Torczon]

- A program that translates an *executable* program in one language into an *executable* program in another language
- The compiler should improve the program, *in some way*



- **Was ist ein Interpreter?** [laut Cooper&Torczon]

- A program that reads an *executable* program and produces the results of executing that program



Beispiele

Sprache	Compiler		Interpreter	
	Maschinencode	VM-Code*	direkt	Zwischencode
C/C++	✓			
C#		✓		
Delphi/Pascal	✓**	✓		
Java/Scala		✓		✓ (JIT)
JavaScript			✓ (JIT)§	
Lisp			✓**	✓
Python				✓
PHP				✓

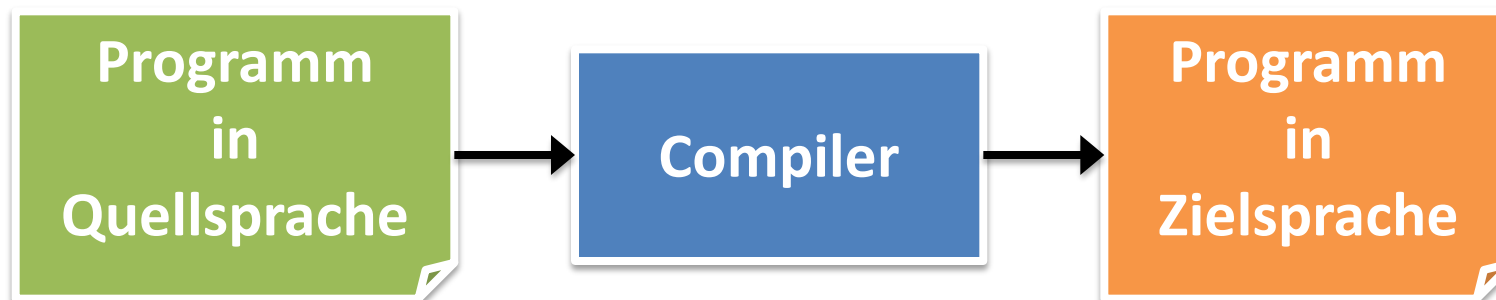
*VM-Code wird in der Regel interpretiert ** frühe Versionen

§ Google V8: JIT in nativen Code

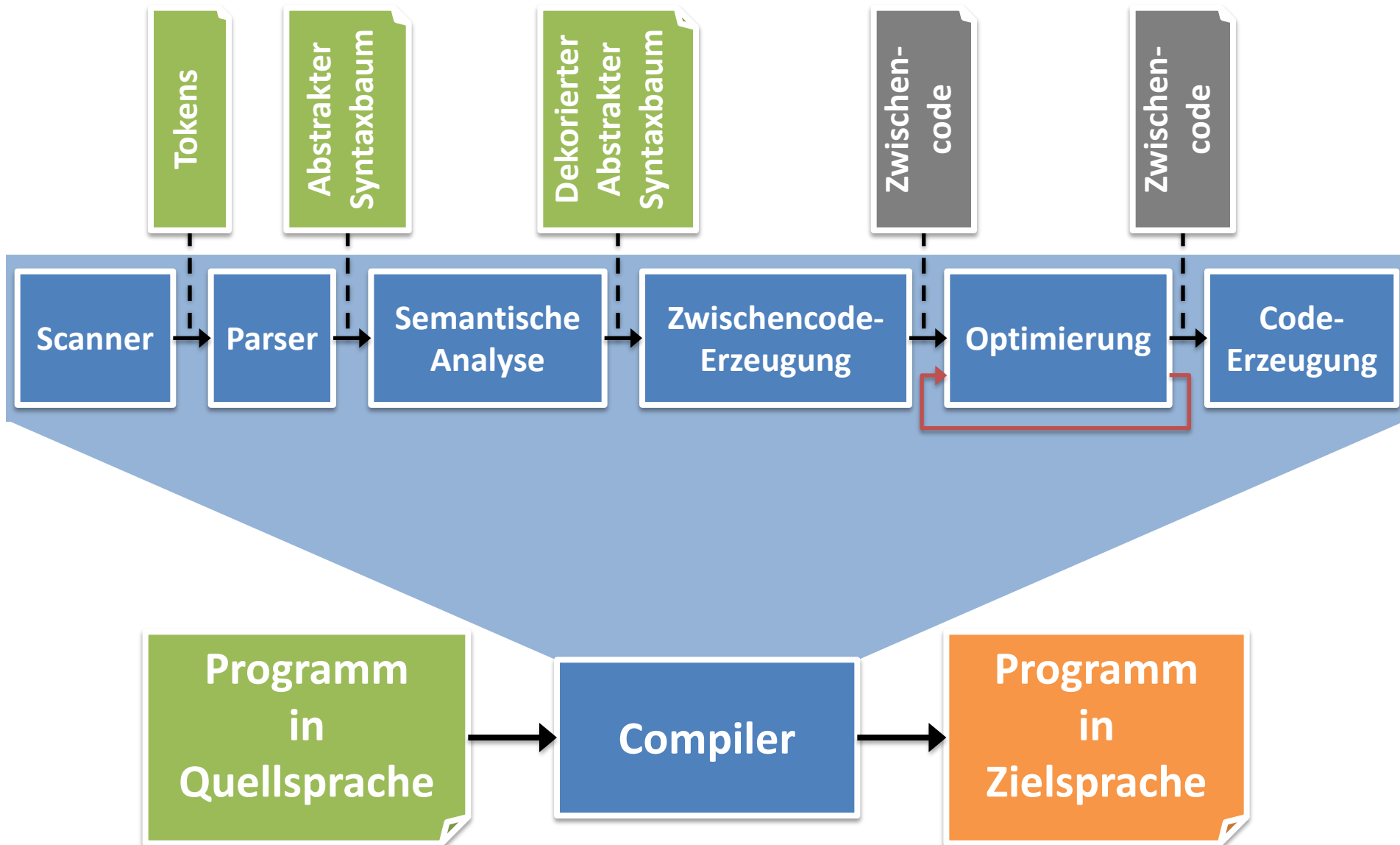
Struktur eines Compilers

Höhere Programmiersprache,
z.B. Java, C, C++, Prolog, ML

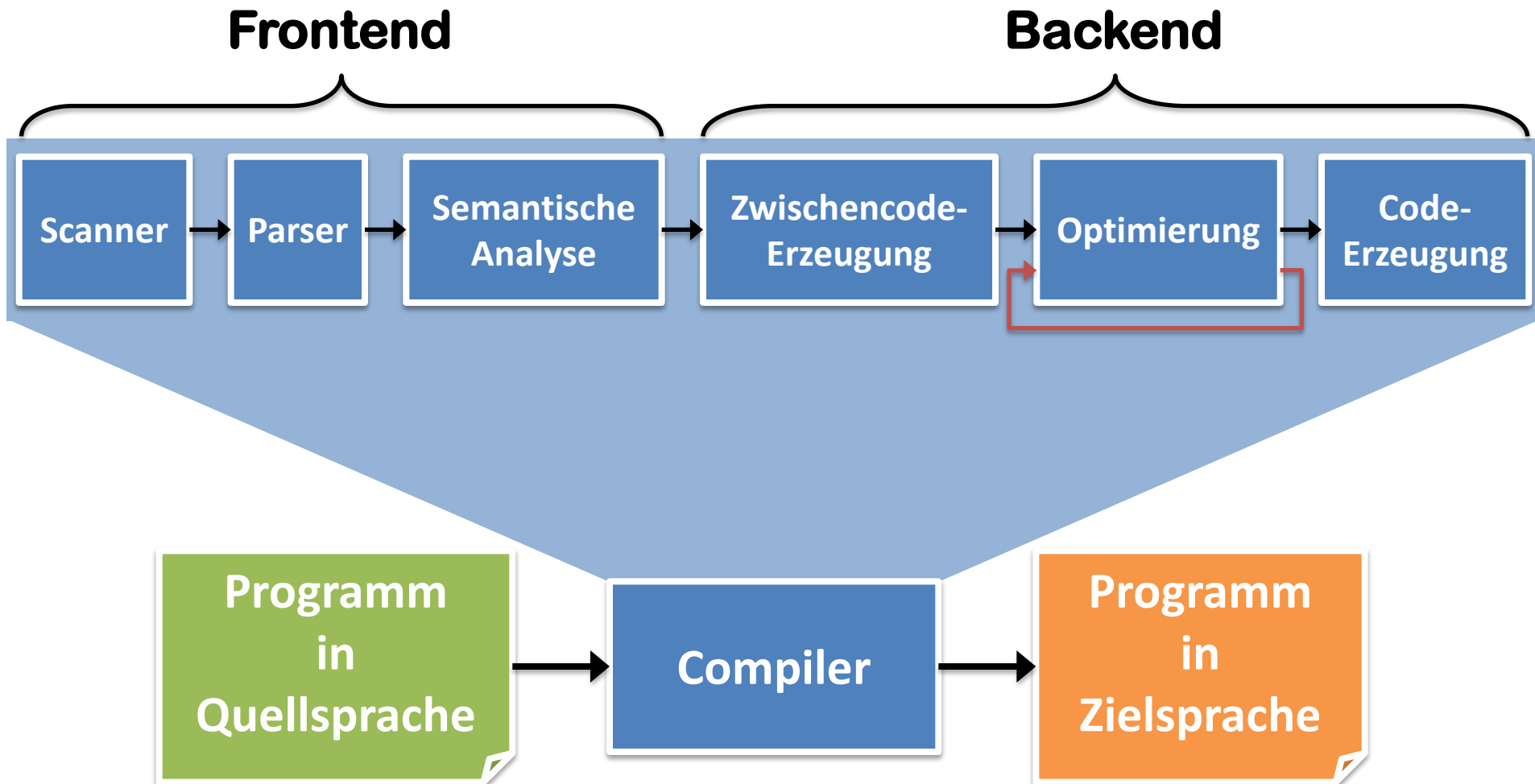
Maschinensprache,
z.B. x86, mc68000, Sparc



Struktur eines Compilers



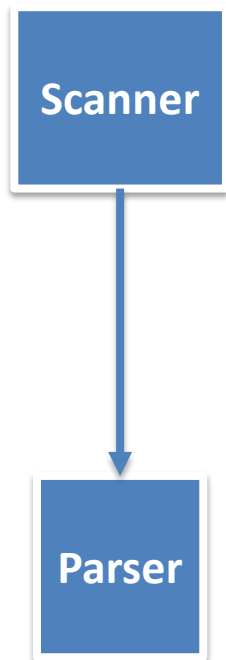
Struktur eines Compilers



Struktur eines Compilers

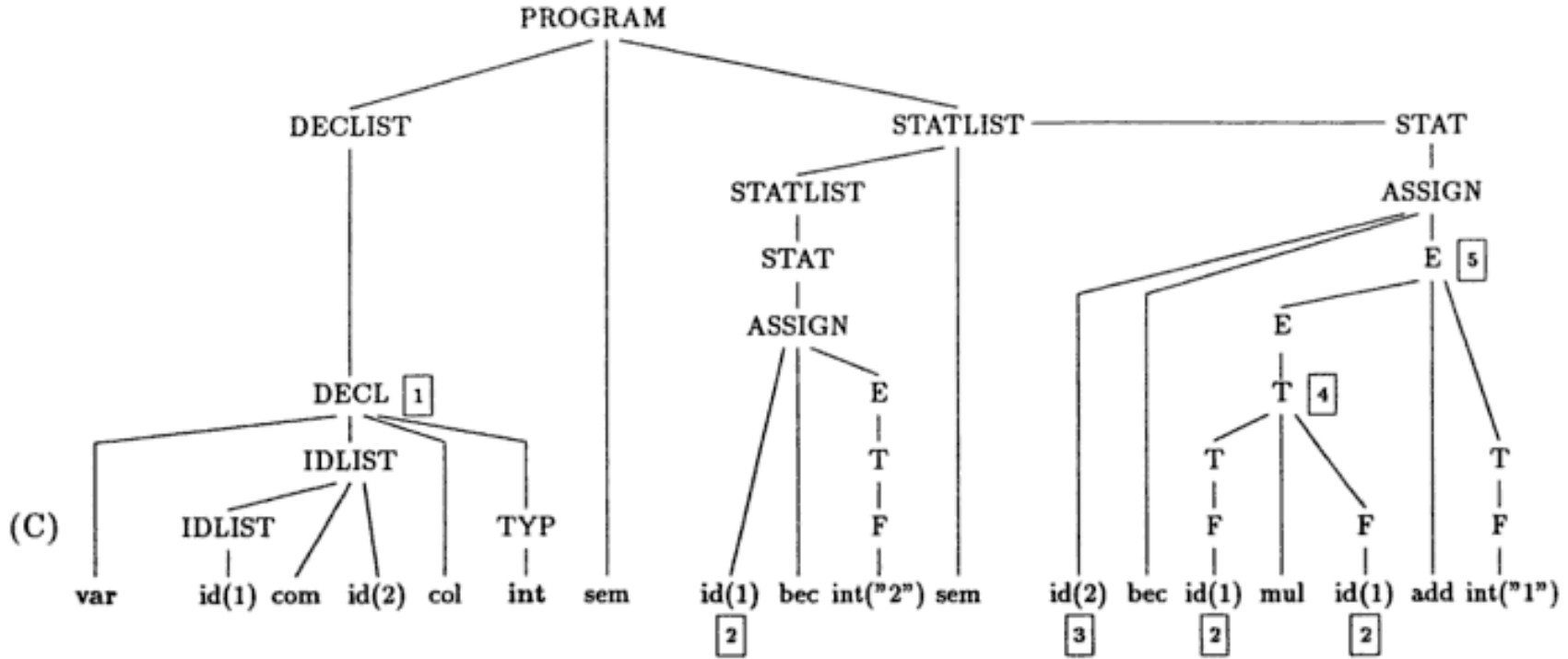
Frontend

- Scanner (Tokenizer, Lexer)
 - Lexikalische Analyse
 - Zerlegung des Quelltexts in logische Einheiten
 - Reguläre Ausdrücke, endliche Automaten
- Parser
 - Syntaktische Analyse
 - Erzeugen eines abstrakten Syntaxbaums
 - Kontextfreie Grammatiken, Kellerautomaten



Struktur eines Compilers

Beispiel



(C) Syntaktische Analyse

(C)

(D)

```
1 (id(1),(var,int))
  (id(2),(var,int))
```

2 (var,int)

3 (var,int)

4 int 5 int

(E)

```
1 (id(1),(var,int,0))
  (id(2),(var,int,0))
```

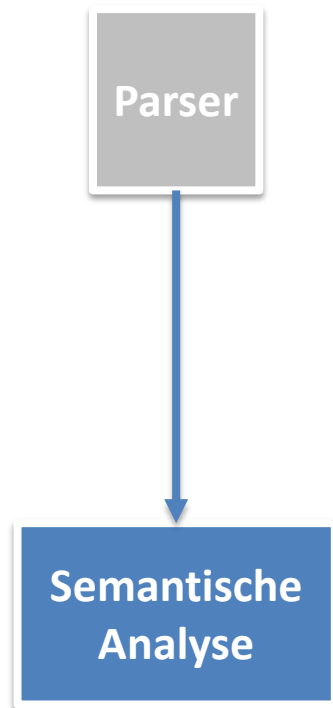
2 (var,int,0)

3 (var,int,1)

Quelle: Übersetzerbau, Wilhelm & Maurer, 1992

Struktur eines Compilers

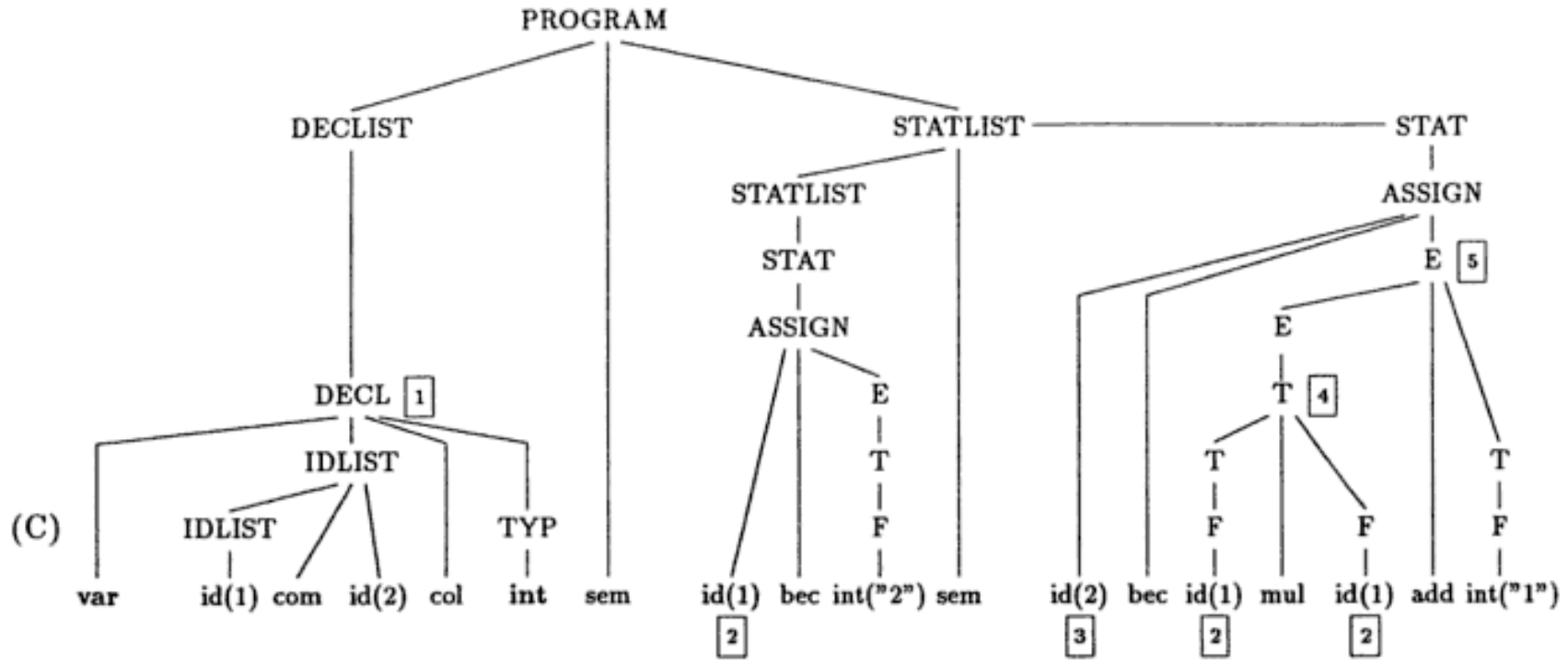
Frontend



- Semantische Analyse
 - Überprüfung der *statischen* Semantik
 - Deklarationen, Typkompatibilität, etc.
 - Attributieren/Dekorieren des abstrakten Syntaxbaums
 - Attributgrammatiken = kontextfreie Grammatiken, die um Attribute und Regeln für diese erweitert wurden
 - Nichtterminale durch zusätzliche Informationen erweitern

Struktur eines Compilers

Beispiel



(C) Syntaktische Analyse

(C)

(D) Semantische Analyse

(D)

1 (id(1),(var,int))
 (id(2),(var,int))
 2 (var,int)
 3 (var,int)
 4 int
 5 int

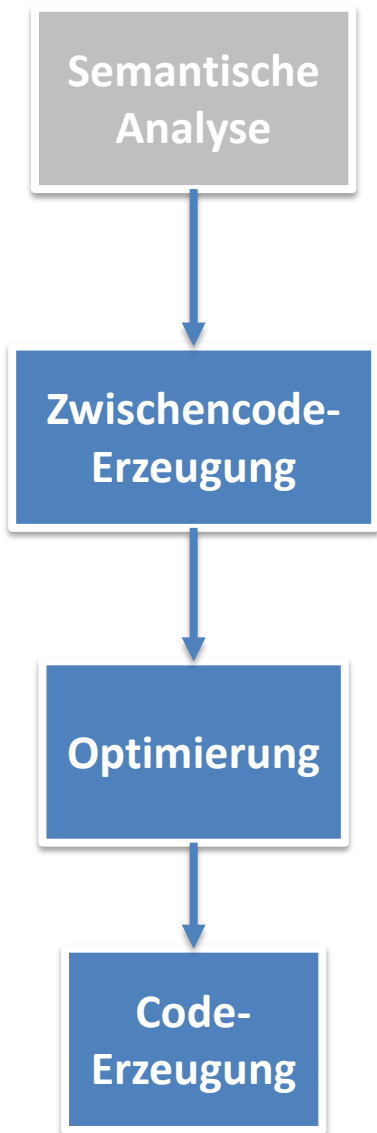
(E) Adresszuordnung

(E)

1 (id(1),(var,int,0))
 (id(2),(var,int,0))
 2 (var,int,0)
 3 (var,int,1)

Struktur eines Compilers

Backend

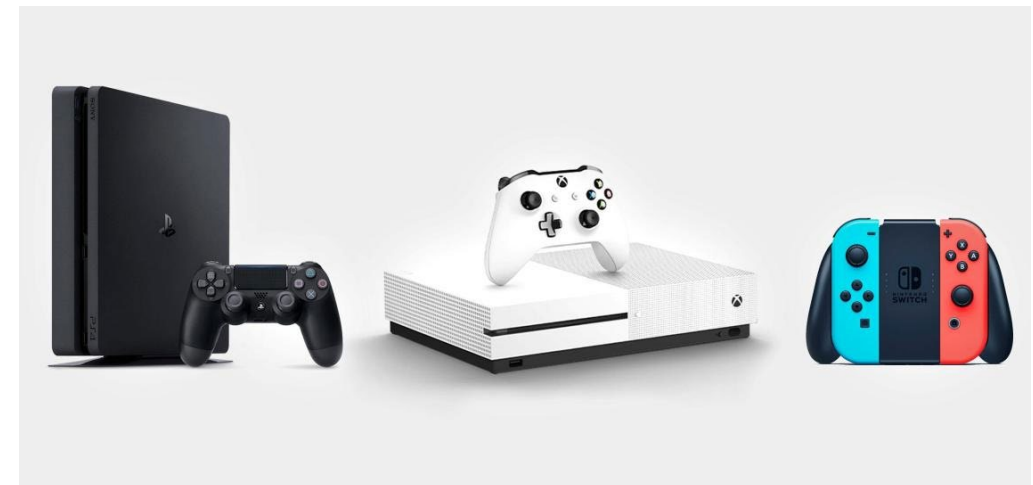
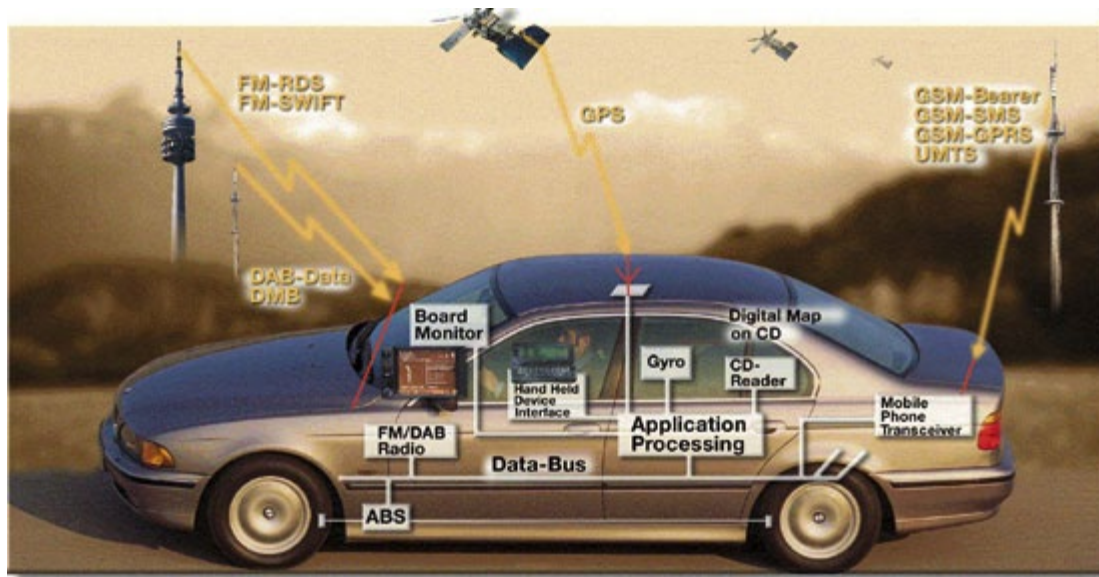
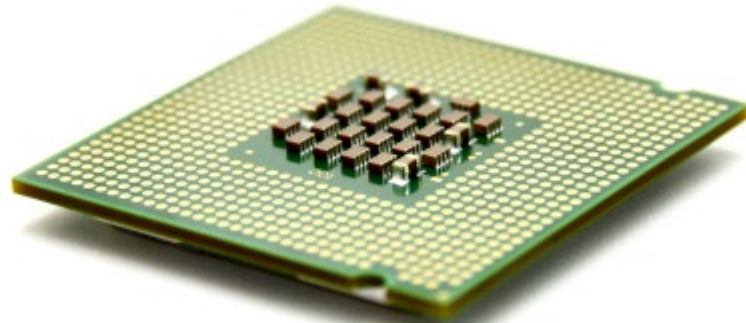


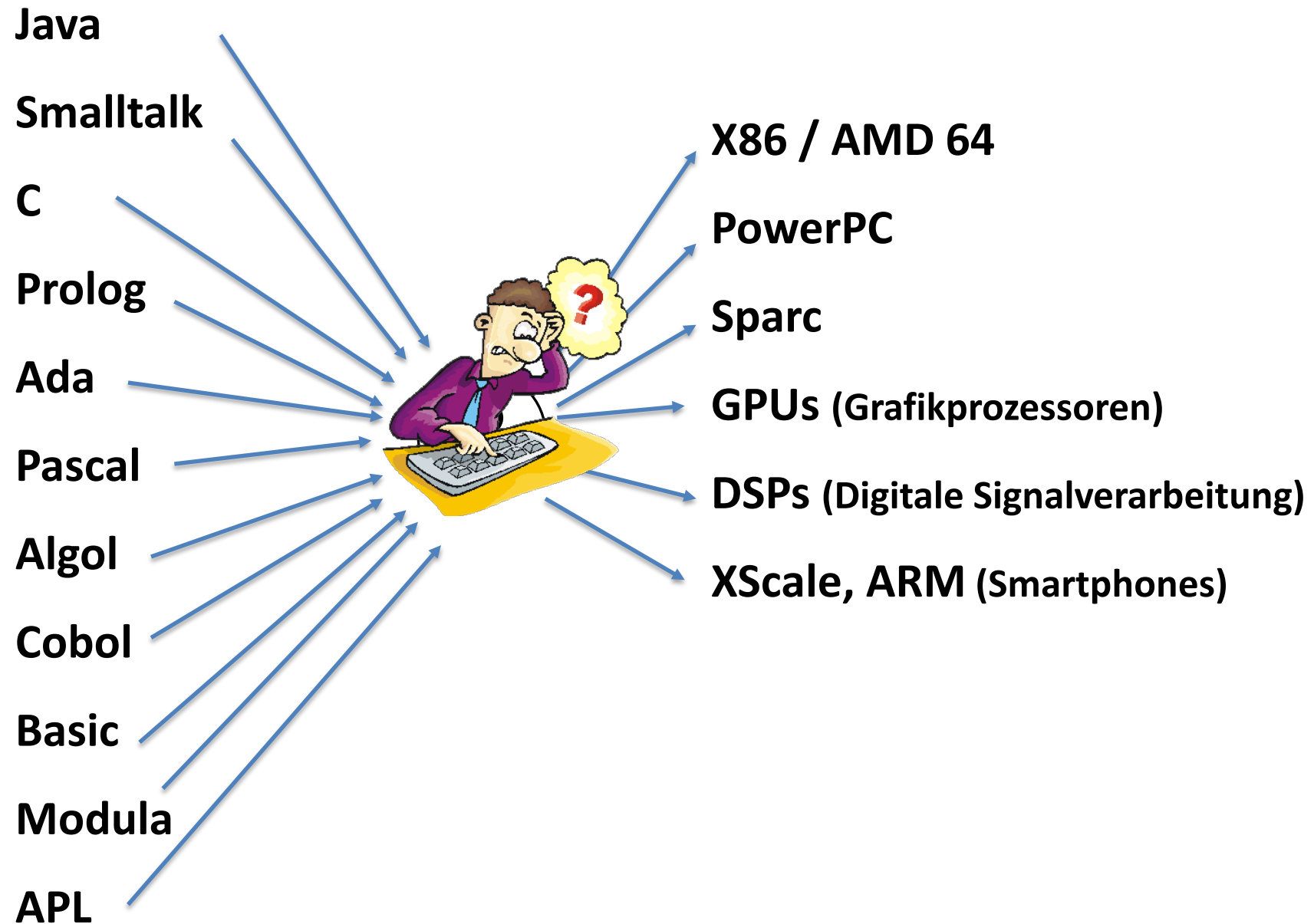
- Erzeugung von Zwischencode
 - relativ maschinennah
 - flexibel (Quellsprachen ↔ Zielplattformen)
- Optimierung des Zwischencodes
 - Verbesserungen von Laufzeit und Speicherbedarf
 - Maschinenbefehle einsparen, Schleifen optimieren, etc.
- Codeerzeugung
 - Erzeugen des ausführbaren Maschinencodes

Das Programmier- sprachenbabel



Vielfalt an Prozessoren

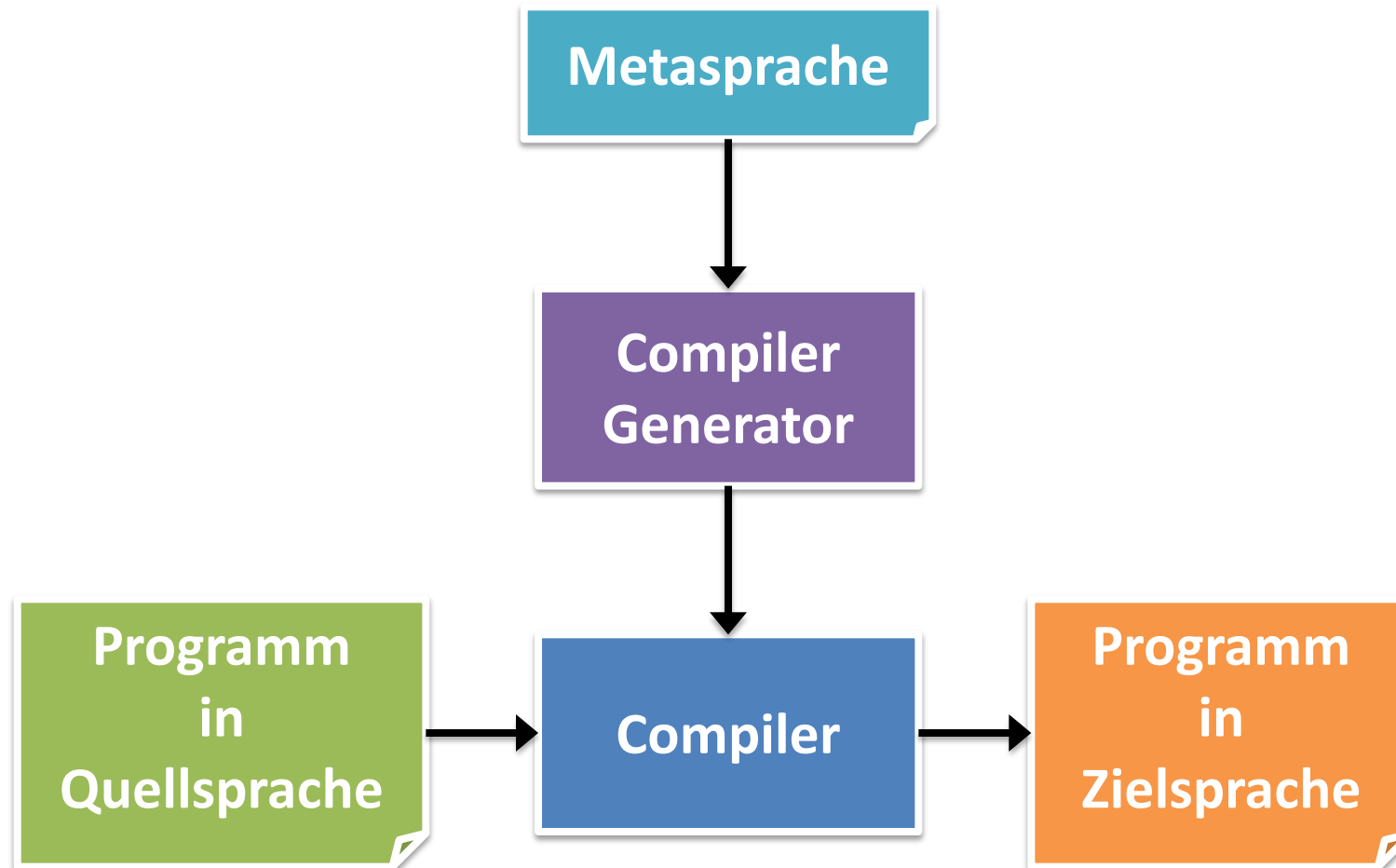




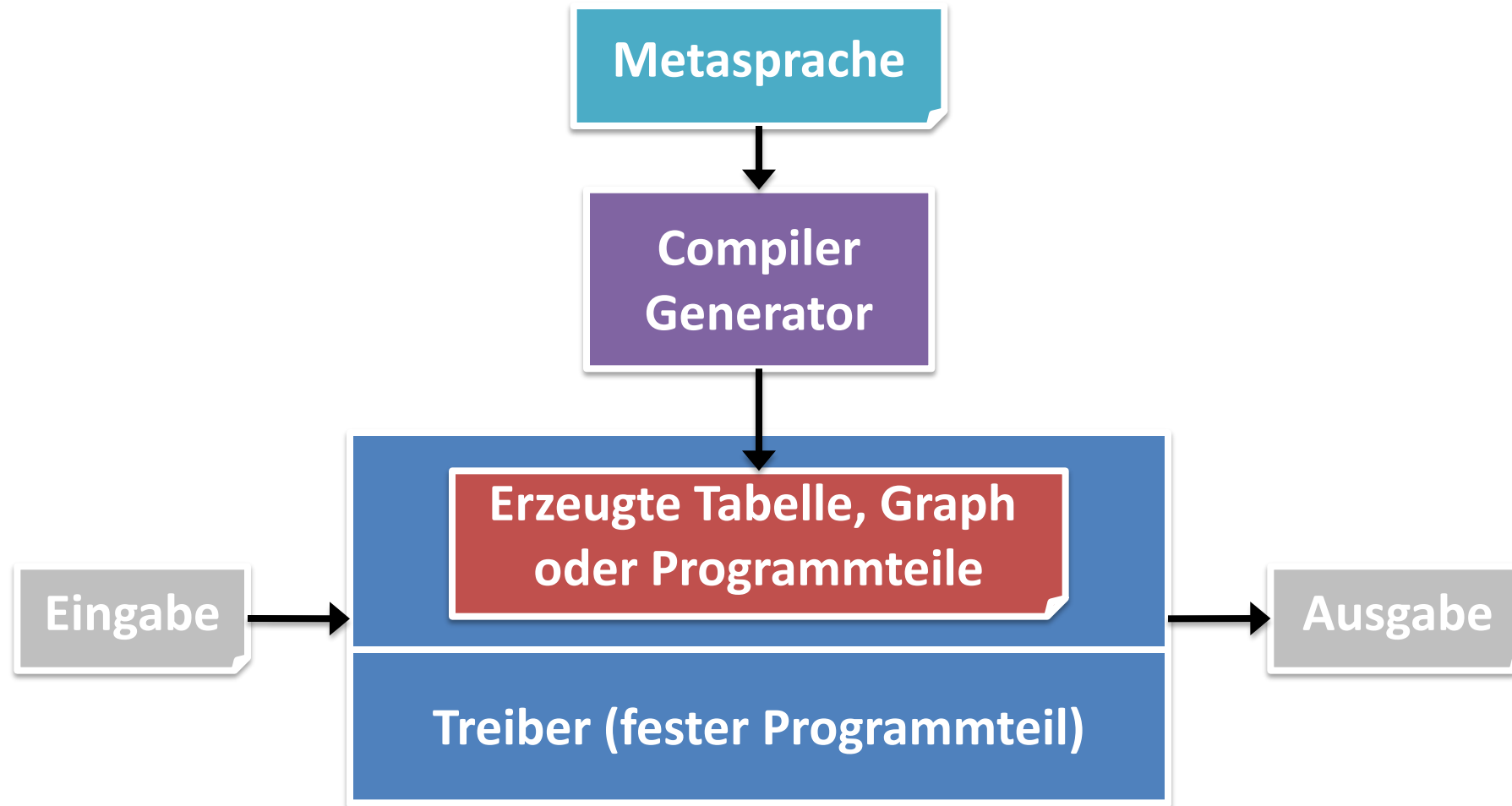
Idee/Traum der Compilerbauer

- N Quellsprachen, M Zielarchitekturen
 $\Rightarrow N * M$ Compiler werden benötigt
- Früher
 - $N * M$ Compiler werden von Hand programmiert
- Heute
 - Teilweise durch Generatoren vereinfacht
- Idealvorstellung
 - N Quellsprachbeschreibungen werden spezifiziert
 - M Zielarchitekturbeschreibungen werden spezifiziert
 - Also: $N + M$ Spezifikationen von Hand
 - $N * M$ Compiler werden daraus generiert

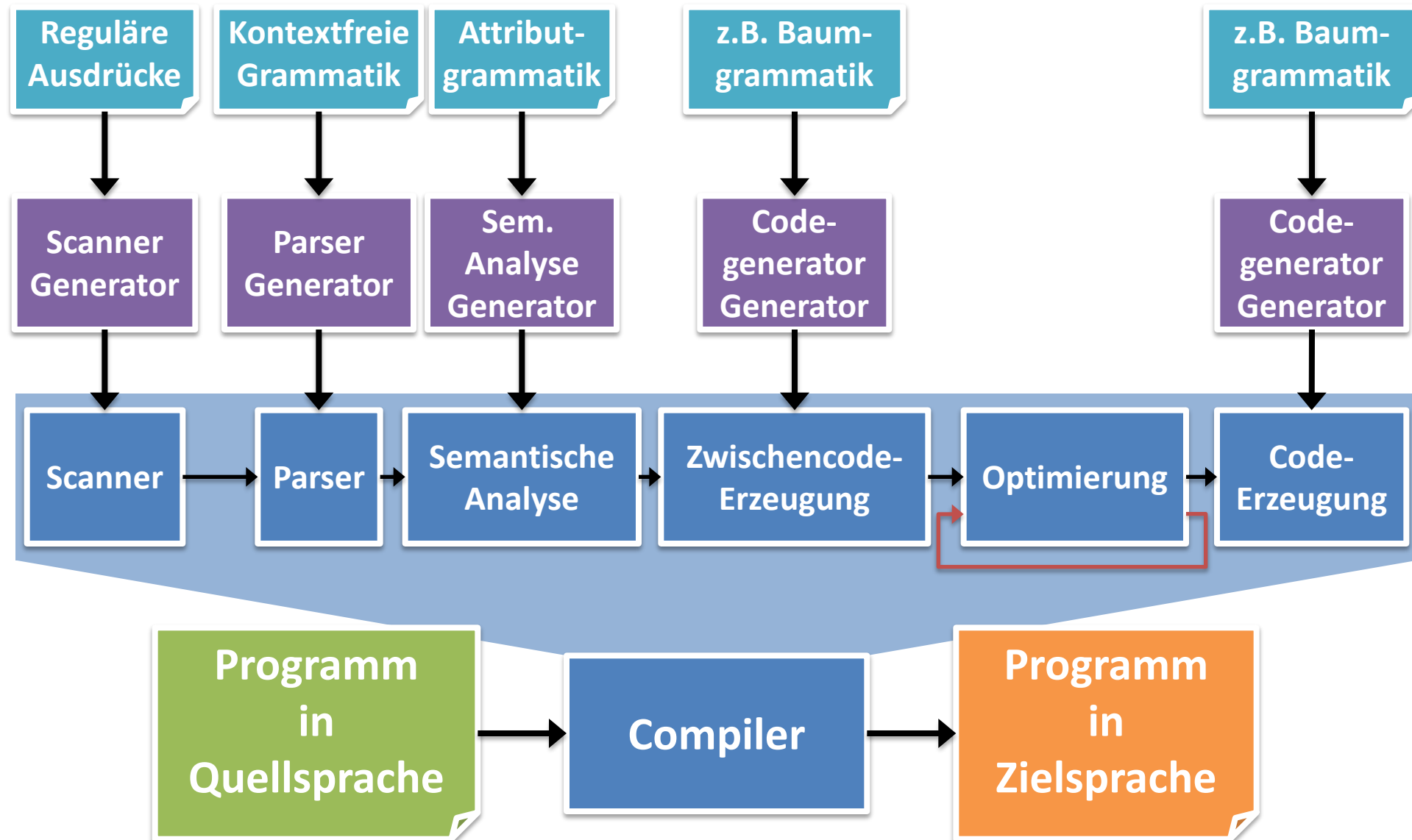
Generierung von Compilern



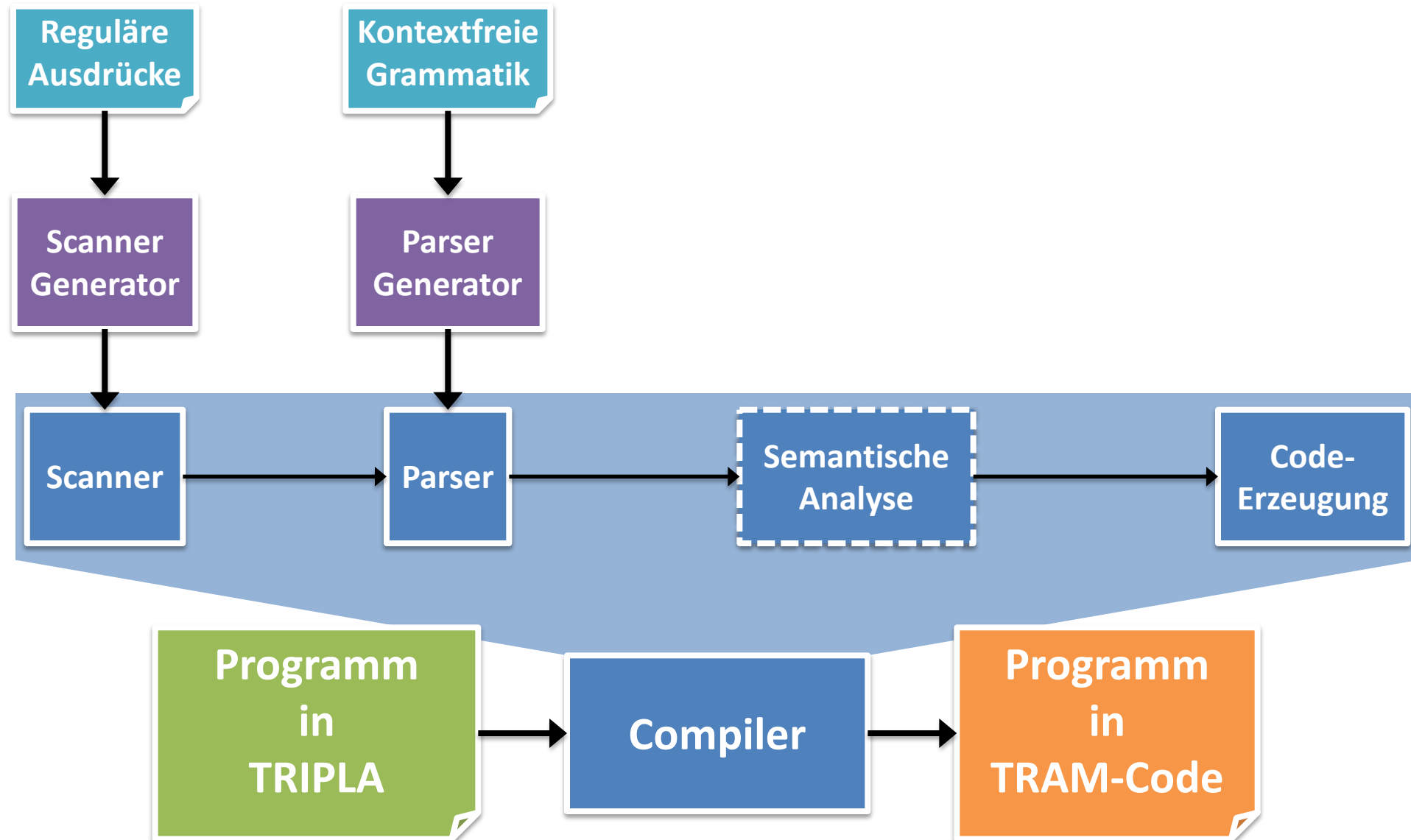
Typische Funktionsweise von Generatoren



Generierung von Compilern



Projekt: Trierer Programmiersprache



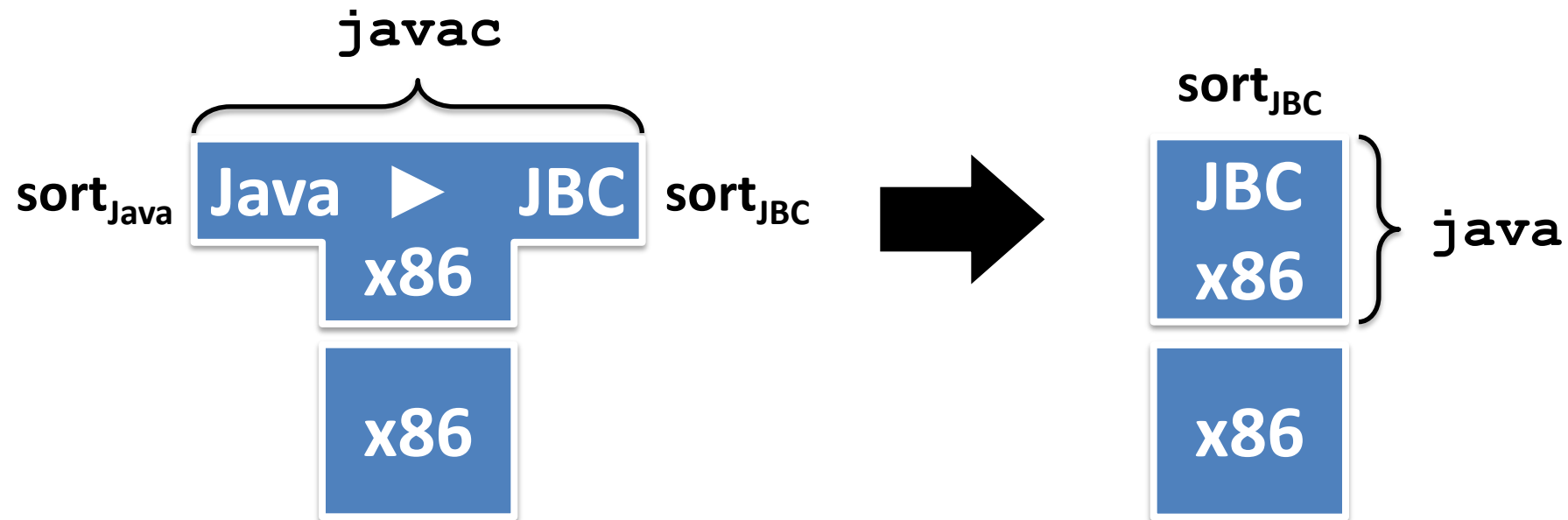
T-Diagramme

- Zusammenspiel von Interpretern und Compilern lässt sich mit Hilfe einer grafischen Notation, den **T-Diagrammen (tombstone)**, veranschaulichen.



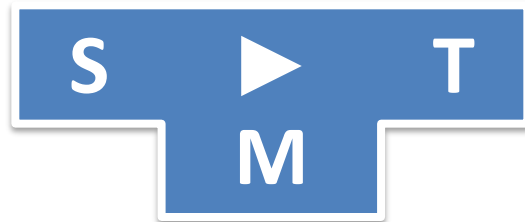
T-Diagramme: Compiler

- **Beispiel:** Java (JDK, SDK)
 - **javac**: Java to Java byte code (JBC) compiler
 - **java**: Java Virtual Machine byte code interpreter



Wie implementiert man einen Compiler?

- Man programmiert ihn direkt in Maschinensprache (native compiler), also:

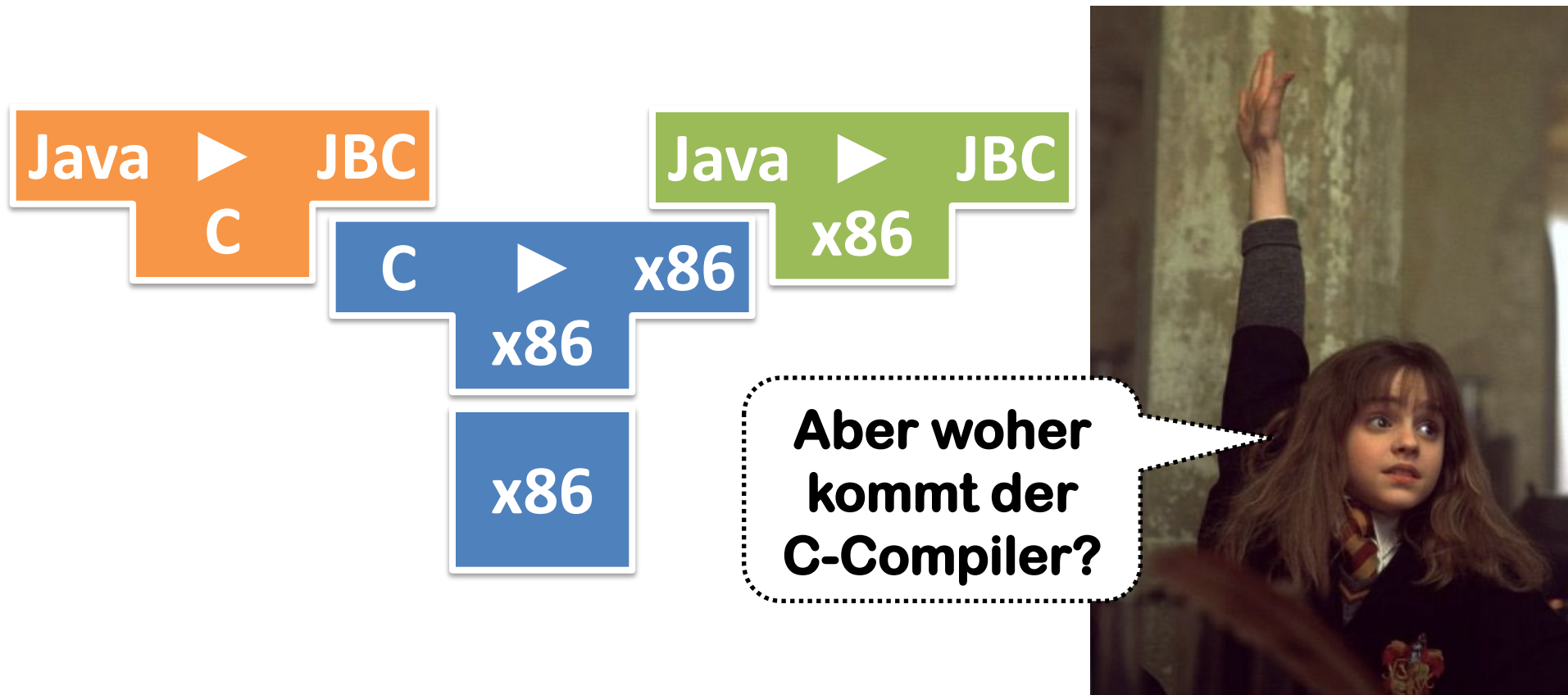


**Zu viel
Arbeit!**



Wie implementiert man einen Compiler?

- Schreibe Compiler in einer einfacheren Programmiersprache:



Wie implementiert man einen Compiler?

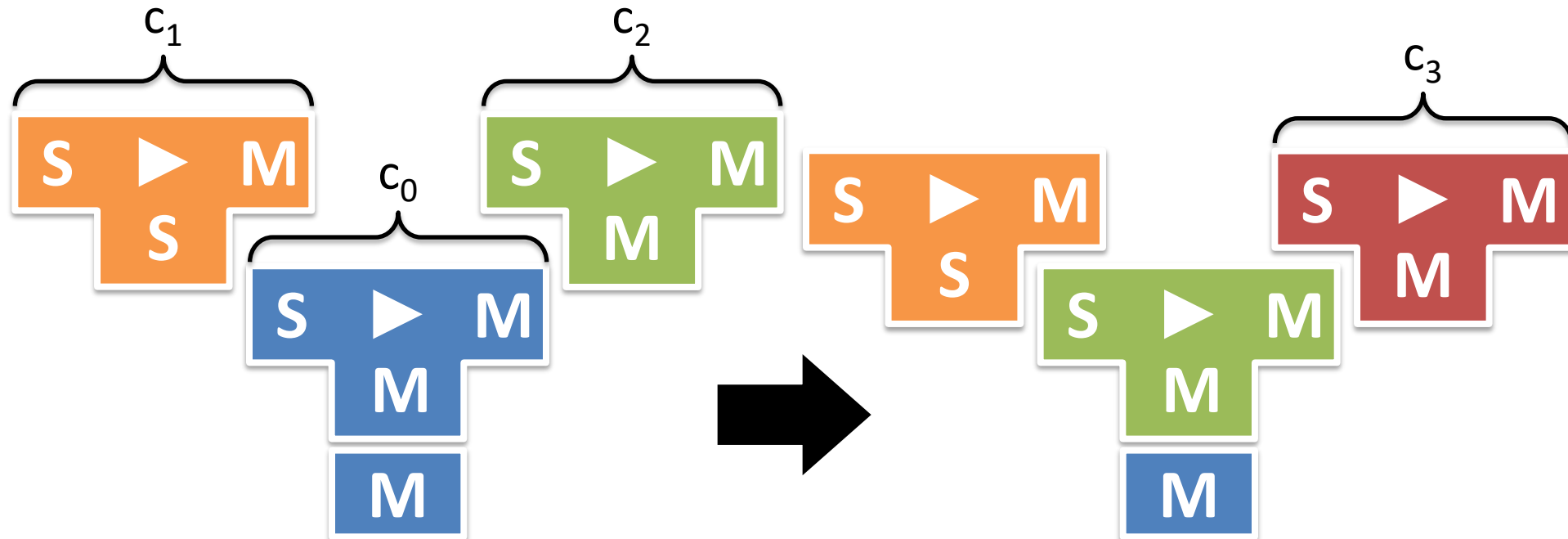
- Schreibe Compiler für eine Sprache L in der Sprache L selbst und verwende *Bootstrapping*.



Wie implementiert man einen Compiler?

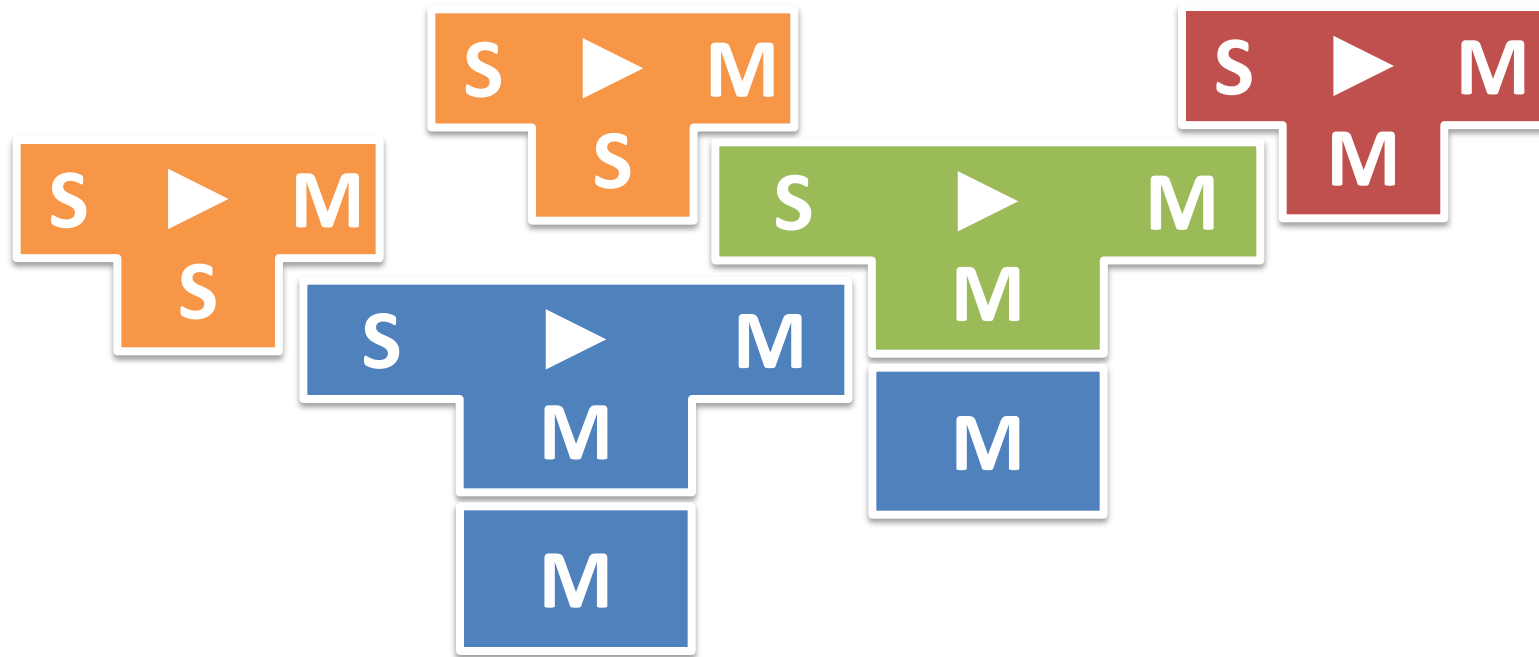
- **Bootstrapping**

- Schreibe einen einfachen, nativen Compiler c_0 für S in der Maschinensprache
- Schreibe einen besseren Compiler c_1 in der Sprache S .
- Verwende Compiler c_0 , um Compiler c_1 zu übersetzen, damit erhält Du einen besseren nativen Compiler c_2 .
- Verwende c_2 , um c_1 zu übersetzen, um einen noch besseren nativen Compiler c_3 zu erhalten.



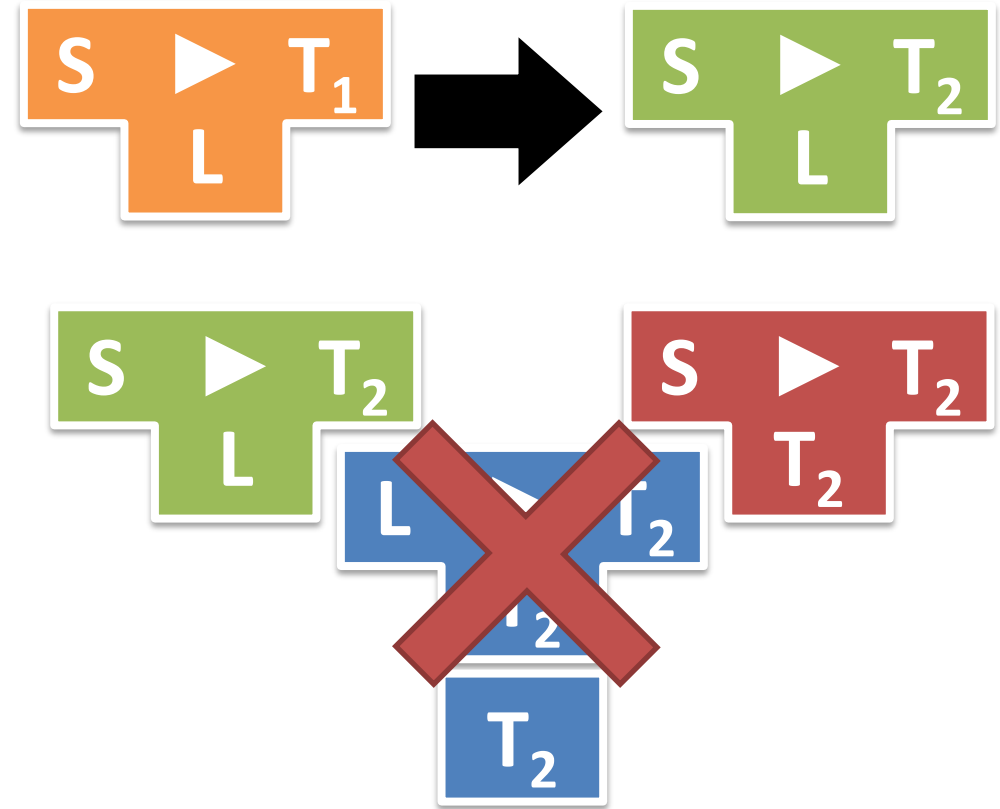
Wie implementiert man einen Compiler?

- **Bootstrapping (in einem T-Diagramm)**



Wie portiert man einen Compiler?

- Passe das Backend an die neue Maschine an:
- Übersetze mit einem nativen Compiler für L auf der neuen Maschine:



**Na, dann hilft nur der
Crosscompiler
Zauber?**

**Und was, wenn man auf
der neuen Maschine noch
gar keinen Compiler hat ?**

Wie portiert man einen Compiler?

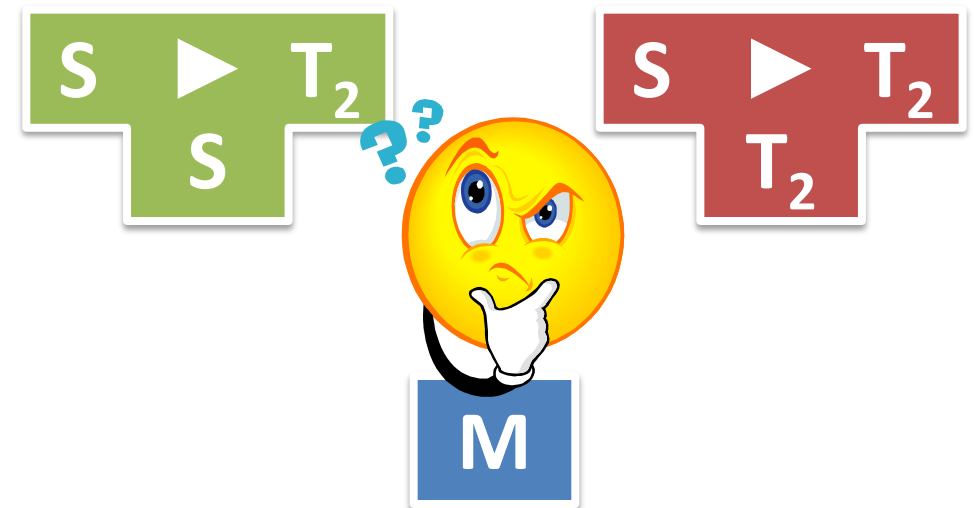
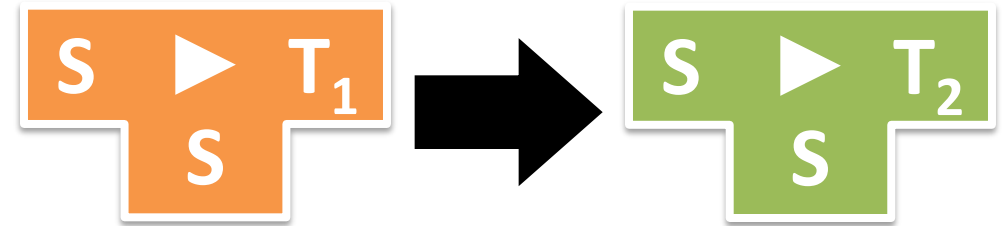
- Ein *Cross-Compiler* kompiliert eine Quellsprache in eine Zielsprache, die nicht der Sprache entspricht, in der der Compiler ausgeführt wird.



T ≠ M

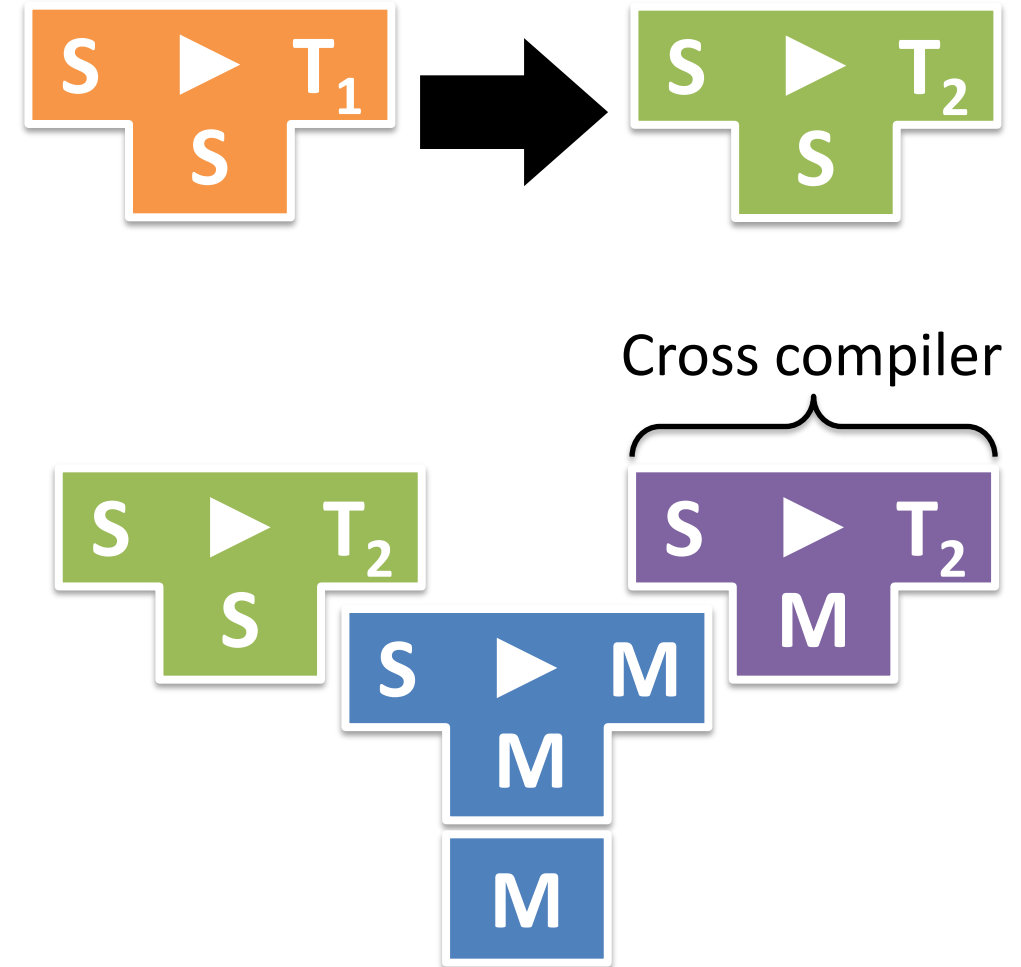
Wie portiert man einen Compiler?

- Voraussetzung ein in einer Sprache S geschriebener Compiler für S. Passe das Backend an die neue Maschine an:



Wie portiert man einen Compiler?

- Voraussetzung ein in einer Sprache S geschriebener Compiler für S. Passe das Backend an die neue Maschine an:
- Übersetze mit einem nativen Compiler für S auf irgendeiner Maschine M:





**Man benötigt einen nativen
Compiler für S, alle anderen können
dann per Cross-Compilation
erzeugt werden.**

